

Term Project Report (Computer Graphics)

Title: Falling Simulator

학번: 2022148079 이름: 이진형

학번: 2023175001 이름: 강희수

1. 요약: Falling Simulator는 Three.js 3D 그래픽 라이브러리와 Rapier 3D 물리 엔진을 활용하여 구현한 실시간 물리 시뮬레이션 애플리케이션입니다. 사용자는 중력 연산과 착지 충격량 연산을 통해 캐릭터가 빌딩 옥상에서 뛰어내리는 과정을 관찰할 수 있습니다. 낙하 시 빌딩의 높이, 캐릭터의 질량, 사망 임계 충격량을 조절하여 여러 상황을 시뮬레이션 할 수 있으며 낙하산, 트램폴린, 윈슈트, 대나무 헬리콥터, 파괴 블록 스택, 웹 슈팅 등의 도구를 설치하여 그 효과를 확인할 수 있습니다. 또한 지구, 달, 화성, 목성, 무중력의 다섯 가지 중력 환경을 선택할 수 있으며, 환경에 따라 중력 값뿐만 아니라 바닥 지형 텍스처와 대기 Fog 색상, 원경 표현 범위가 실시간으로 변경되어 더욱 몰입감 있는 시뮬레이션을 제공합니다.

2. 사용법:

- 마우스 조작: 드래그를 통해 카메라 궤적을 회전할 수 있습니다.
- WASD 또는 방향키: 지상에서는 캐릭터 이동, 수중에서는 수영 방향 제어, 공중에서는 미세한 수평 공중 제어를 수행합니다. '윈슈트' 장착 시에는 전진 가속 및 기울이는 방향을 제어하며, '대나무 헬리콥터' 장착 시에는 수평 비행 방향을 제어합니다.
- Spacebar: 지상/옥상에 서 있을 때는 '낙하 시작(Jump)'을 수행합니다. 공중에서 '낙하산' 장착 시에는 낙하산을 펴고, '윈슈트' 장착 시에는 날개를 펴서 활공을 시작합니다. '대나무 헬리콥터' 장착 시에는 Spacebar를 유지하여 상승 기류를 타고 하늘로 날아오릅니다.
- Key E: '웹 슈팅' 장착 상태에서 거미줄을 발사하여 연결하거나, 다시 눌러 끊어서 관성 비행으로 돌입합니다.
- 행동 제어: '낙하 시작', '리스폰' 버튼을 클릭해서 조작할 수 있습니다.
- GUI 조작 패널:
 -  맵 선택: '도시(City)' 및 '바다(Sea)' 지형을 실시간으로 전환합니다.
 -  빌딩 높이 (m): 시작 빌딩 높이를 0m에서 828m(세계에서 가장 높은 건물)까지 실시간으로 조절합니다.
 -  캐릭터 질량 (kg): 캐릭터 무게를 조절합니다.
 -  사망 임계 충격량: 사망 기준치를 조절합니다.
 -  장착 도구: '없음(None)', '낙하산(Parachute)', '트램폴린(Trampoline)', '윈슈트(Wingsuit)', '대나무 헬리콥터', '파괴 블록 스택', '웹 슈팅(Web Shooting)'을 장착합니다. 트램폴린의 경우 트램폴린 정밀 조정 설정에 위치 및 반경을 조절하고, 파괴 블록 스택의 경우 블록 스택 세부 조정 설정에서 배치 개수(층수)를 조절합니다.
 -  중력 환경: '지구', '달', '화성', '목성', '무중력' 프리셋을 실시간 변경하여 물리 상수와 환경을 전환합니다.



3. 기능 구현 표: (구현 기능 별로 구현 source code 위치를 표기)

	기능	구현위치	비고
1	Texture mapping	main.js (lines 417-435, 1026-1129)	도시, 바다, 달, 화성, 목성 지면, 시작 빌딩 벽면, 윈슈트 날개, 대나무 헬리콥터, 하늘 및 우주 환경 텍스처 매핑을 구현함.
2	Lighting	main.js (lines 388-406)	AmbientLight, HemisphereLight, DirectionalLight을 추가하여 3D 공간 내 자연스러운 조명 효과를 만듦.
3	Physically based Animation	main.js (lines 351, 991-994, 1275-1333, 1590-1720, 1736-1827)	Rapier 3D 물리 엔진 기반 중력 가속도 프리셋, 실시간 중력 변경, 충격량 완화/판정 연산을 구현함. 파괴 블록 충돌 시 관통/파괴 및 윈슈팅 시 Rope Joint를 활용한 진자 운동 물리 시뮬레이션을 구현함.
4	Shadows	main.js (lines 339-340, 358-368, 918-921, 1072, 1114, 1127, 1199)	renderer.shadowMap 활성화 및 PCFSOFTShadowMap으로 부드러운 그림자를 설정함. 캐릭터, 시작/배경 빌딩, 윈슈트 날개, 헬리콥터, 파편 메쉬에 castShadow/receiveShadow 속성을 양방향 바인딩하여 정밀한 동적 입체 그림자를 구현함.
5	Keyframe Animation	main.js (lines 1194, 1295-1296, 1369-1385, 1505-1517, 1586, 1992-1996, 2008-2015, 2056-2062, 2068-2072)	THREE.AnimationMixer를 활용해 프레임 타임 델타(dt)에 맞춘 애니메이션 상태 업데이트함. 이동 속도(horizontalSpeed)와 발 움직임을 동기화하기 위한 timeScale 가중치 보정 및 상황별(대기, 걷기, 낙하, 사망, 회복, 입수, 수영) 애니메이션 전환함

6	Skeletal Animation	main.js (lines 1182-1262)	FBXLoader를 활용한 Timmy 캐릭터 모델 Bone 구조 로드함. Idle, Walking, Falling, FallDie, FallLive, WaterStand, Swimming 애니메이션을 각각 상황별로 블렌딩 전환함.
7	User Interaction & GUI	main.js (lines 171-292, 297-311, 1083-1180)	lil-gui를 활용한 빌딩 높이, 캐릭터 질량, 맵 종류, 장착 도구, 트램펄린 좌표/반경, 블록 스택 개수, 중력 프리셋 등의 실시간 파라미터를 제어함. WASD, Spacebar, Key E 등의 키보드 입력 바인딩 및 GUI 포커스 자동 회수 처리함.

4. 기타 사용 기능 (수업시간에 배우지 않은 기능들)

- GLTFLoader를 통한 마천루 모델 로드 및 조밀한 배경 도시 배치: 기존 예제에서는 FBXLoader만 학습하였으나, 본 프로젝트에서는 도시 마천루 19종 모델의 배치와 성능 최적화를 위해 GLTFLoader를 도입했습니다. 도로 구획을 지켜 건물들이 겹치지 않게 격자 기반 랜덤 배치 알고리즘을 설계 및 적용했습니다.
- 물속 부력 시뮬레이션 및 유체 마찰 역학: 기존 예제에서는 단순 중력 및 구체/박스 충돌 연산만 다루었으나, 바다 맵 구성을 위해 수중 입수 시 캐릭터의 깊이(depth)에 비례하는 아르키메데스 부력 원리를 기반으로 부력 모델을 구현하였습니다($\text{buoyancy} = \text{depth} * 25.0$). 또한, 수평 이동 시 물의 유체 마찰 저항(수평 속도에 0.98 감쇄)을 적용했으며 입수 직후 예상 최대 잠수 깊이(targetMaxDepth)를 실시간 산출하여 UI에 표시합니다.
- 트램펄린 탄성 완화 진동: 트램펄린 착지 시 중력 가속도를 흡수하는 탄성 처리를 구현하기 위해, 충격을 85% 완화 감쇄시키고($\text{impulse} * 0.15$), 착지 후 타이밍을 감지해 감쇄 하강 속도에 비례한 리바운드 탄성 반등 상승을 물리 속도로 부여했습니다. 충격이 너무 과도하면 파손으로 추락 사망하는 판정 또한 구현했습니다.
- 캔버스 기반 낙하산 생성: 예제처럼 이미지 파일을 텍스처로 불러오는 대신, HTML5 캔버스를 이용하여 주황색/흰색 줄무늬 패턴을 실시간으로 생성하였습니다. 16개의 세로 구간을 반복문으로 순회하며 낙하산 무늬를 만든 후, THREE.CanvasTexture로 변환하여 3D 모델에 적용하였습니다. 이러한 방식으로 별도의 이미지 리소스 없이 원하는 패턴을 동적으로 생성하였습니다.
- 포커스 recapture 시스템: GUI 마우스 조작 시 브라우저 포커스가 GUI 엘리먼트에 강제로 묶여 WASD 캐릭터 키보드 조작이 일시 차단되는 인터랙션 충돌 문제가 발생하였습니다. 따라서 GUI 조작 후 `document.activeElement.blur()`를 호출하여 포커스를 자동 해제하였습니다.
- Rapier 3D impulse joint 기반 진자 운동 구현: RAPIER.JointData.rope()와 createImpulseJoint를 활용해 캐릭터와 앵커 간의 유연한 거미줄 연결을 구현하였습니다. 또한 거미줄 해제 시 관성력을 계산하여 관성 비행을 수행하도록 구현하였습니다.

5. 기타

- 행성 프리셋별 대기 환경 동적 융합: 지구(노을 안개), 달(진공 우주), 화성(주황 안개), 목성, 무중력 등 우주적 연출을 텍스처 변경뿐만 아니라 Fog의 범위와 색상 변경을 통해 시각적인 부분을 구현했습니다.